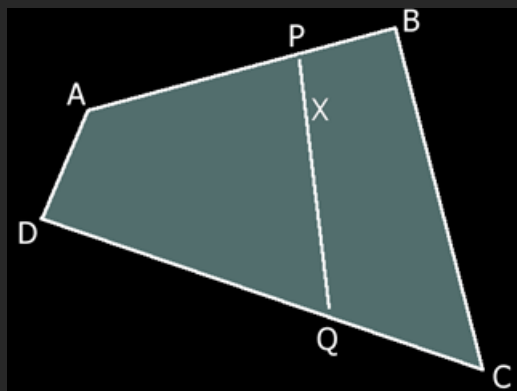


Inigo Quilez :: articles :: inverse bilinear interpolation - 2010



Intro

You have probably done bilinear interpolation in a quadrilateral more than once already. Say you have a random quadrilateral with vertices **A**, **B**, **C** and **D**, in any number of dimensions. Then any point **X** inside the quadrilateral can be defined by a couple of coordinates u and v such that the first interpolates across any two opposite edges of the quad to produce two points **P** and **Q**, and such that the second one linearly interpolates again across the line that joins **P** and **Q**. The nice thing is that the same rules that are used to arrive to **X** can be used for the any data stored in the vertices (say, texture coordinates or colors) so that one gets the bilinearly interpolated data values at the point **X**.



The math

The formula that tells us how to get **X** is very simple:

P is the linear interpolation of **A** and **B** in u : $P = A + (B-A) \cdot u$

Q is the linear interpolation of **D** and **C** in u : $Q = D + (C-D) \cdot u$

X is the linear interpolation of **P** and **Q** in v : $X = P + (Q-P) \cdot v$

therefore

$$\mathbf{X}(u,v) = \mathbf{A} + (\mathbf{B}-\mathbf{A}) \cdot u + (\mathbf{D}-\mathbf{A}) \cdot v + (\mathbf{A}-\mathbf{B}+\mathbf{C}-\mathbf{D}) \cdot u \cdot v$$

with u and v running in the $[0..1]$ interval obviously. So far so good. This is old news, really. However, what happens if we now had our quad, we knew the point $\mathbf{X}$ and wanted to extract what are the bilinear parameters u and v ?

Well, the easy answer is that we simply have to solve this equation and isolate our parameters u and v . "One equation and two unknowns!", you might protest. Remember that the equation above actually applies to all the coordinates of the system, meaning that the vertices $\mathbf{A}$ to $\mathbf{D}$ are in fact pairs or triplets of numbers (or vectors in general), so in fact we have lots of equations, as many as dimensions we are working on, for only two unknowns. So the systems is very solvable, and the easy answer happens to be the right one.

To make things easier, let's introduce some intermediate variables, such that:

$$\mathbf{E} = \mathbf{B}-\mathbf{A}$$

$$\mathbf{F} = \mathbf{D}-\mathbf{A}$$

$$\mathbf{G} = \mathbf{A}-\mathbf{B}+\mathbf{C}-\mathbf{D}$$

$$\mathbf{H} = \mathbf{X}-\mathbf{A}$$

so that the bilinear interpolation equation becomes

$$\mathbf{H} = \mathbf{E} \cdot u + \mathbf{F} \cdot v + \mathbf{G} \cdot u \cdot v$$

Now, we choose any two coordinates for our computations. Probably the best choice is to use the 2D plane for which our quad has bigger surface area under orthographic projection. Say we choose axes i and j anyways ($\{i,j\}$ can be $\{x,y\}$, $\{z,x\}$, $\{y,z\}$, ...). Then

$$u = (\mathbf{H}_i - \mathbf{F}_i \cdot v) / (\mathbf{E}_i + \mathbf{G}_i \cdot v)$$

and therefore we can plug it back into the equation to get

$$\mathbf{H}_j \cdot \mathbf{E}_i + \mathbf{H}_j \cdot \mathbf{G}_i \cdot v = \mathbf{E}_j \cdot \mathbf{H}_i - \mathbf{E}_j \cdot \mathbf{F}_i \cdot v + \mathbf{F}_j \cdot \mathbf{E}_i \cdot v + \mathbf{F}_j \cdot \mathbf{G}_i \cdot v^2 + \mathbf{G}_j \cdot \mathbf{H}_i \cdot v - \mathbf{G}_j \cdot \mathbf{F}_i \cdot v^2$$

This is a quadratic equation $k_2 \cdot v^2 + k_1 \cdot v + k_0 = 0$ on v , with coefficients:

$$k_2 = \mathbf{G_i} \cdot \mathbf{F_j} - \mathbf{G_j} \cdot \mathbf{F_i}$$

$$k_1 = \mathbf{E_i} \cdot \mathbf{F_j} - \mathbf{E_j} \cdot \mathbf{F_i} + \mathbf{H_i} \cdot \mathbf{G_j} - \mathbf{H_j} \cdot \mathbf{G_i}$$

$$k_0 = \mathbf{H_i} \cdot \mathbf{E_j} - \mathbf{H_j} \cdot \mathbf{E_i}$$

which are pretty much some surface areas (kind of 2D cross products) of different pairs of edges. The solution is the usual formula for the quadratic equation, where the discriminant $k_1^2 - 4 \cdot k_0 \cdot k_2$ is always positive if **X** is inside the quadrilateral **ABCD**. Then, one has to take the positive or negative square root such that v is in the interval $[0..1]$. Once we have v , u can be computed with the expression used in the first variable isolation step.

Care must be taken in the cases where the quad is actually a perfect rectangle. Then **AB** and **CD** are parallel, and therefore **G** = 0, and it follows $k_2 = 0$ as well. So the equation is basically not a quadratic equation anymore but a simple linear equation of the form $k_0 + k_1 \cdot v = 0$ for which we simply get $v = -k_0/k_1$

The code

Here goes some code for the maths above, specialized for 2d:

```
float cross( in vec2 a, in vec2 b ) { return a.x*b.y - a.y*b.x; }

vec2 invBilinear( in vec2 p, in vec2 a, in vec2 b, in vec2 c, in vec2 d )
{
    vec2 res = vec2(-1.0);

    vec2 e = b-a;
    vec2 f = d-a;
    vec2 g = a-b+c-d;
    vec2 h = p-a;

    float k2 = cross2d( g, f );
    float k1 = cross2d( e, f ) + cross2d( h, g );
    float k0 = cross2d( h, e );

    // if edges are parallel, this is a linear equation
    if( abs(k2)<0.001 )
    {
        res = vec2( (h.x*k1+f.x*k0)/(e.x*k1-g.x*k0), -k0/k1 );
    }
    // otherwise, it's a quadratic
    else
    {
        float w = k1*k1 - 4.0*k0*k2;
```

```
if( w<0.0 ) return vec2(-1.0);
w = sqrt( w );

float ik2 = 0.5/k2;
float v = (-k1 - w)*ik2;
float u = (h.x - f.x*v)/(e.x + g.x*v);

if( u<0.0 || u>1.0 || v<0.0 || v>1.0 )
{
    v = (-k1 + w)*ik2;
    u = (h.x - f.x*v)/(e.x + g.x*v);
}
res = vec2( u, v );
}

return res;
}
```

And here's a realtime running implementation (embedded from <https://www.shadertoy.com/view/lsBSDm>):



inigo quilez - learning computer graphics since 1994